

Proyecto final

Computación en la nube



JUAN DAVID GUZMAN GALLEGO

1.092.850.749.

JAHN CARLOS MARTINEZ VASQUEZ

1.096.032.520

ADRIANA LUCIA MURILLO TASCON

1094967421

Universidad del Quindío
Facultad de ingeniería
Ingeniería de sistemas y computación
Computación en la nube (Profundización)
Octavo semestre 2025-2
03/11/2025
Armenia, Quindío

HTTP como Servicio con DNS en Entorno Local usando Oracle VirtualBox, Apache Web Server y Bind9

En el contexto actual de la computación en la nube y la virtualización de servicios, la capacidad de simular entornos de infraestructura como servicio (**IaaS**) y plataforma como servicio (**PaaS**) se ha convertido en una competencia clave para profesionales en formación. Este proyecto propone la implementación de un entorno local virtualizado que integra tecnologías como Oracle **VirtualBox**, **Apache Web Server** y **Bind9**, con el objetivo de diseñar un servicio administrado de publicación temporal de páginas y aplicaciones web.

La iniciativa parte de la necesidad de comprender y aplicar los principios fundamentales de los servicios **HTTP** y **DNS** en escenarios controlados, replicando el comportamiento de plataformas reales en la nube. En particular, se busca automatizar el aprovisionamiento de instancias del servidor Apache, gestionar dinámicamente los registros DNS mediante Bind9, y ofrecer a los usuarios una interfaz web que facilite la solicitud, despliegue y monitoreo de sus contenidos.

Este entorno permite explorar la interacción entre componentes virtuales, la configuración de redes locales, el manejo de dominios internos, y la automatización de tareas administrativas, consolidando habilidades prácticas en administración de servicios, scripting, y orquestación de infraestructura. El proyecto no solo aborda la problemática de la gestión manual de servicios web en entornos educativos o de laboratorio, sino que también plantea una solución escalable y reproducible que simula el funcionamiento de una plataforma de servicios en la nube.

Objetivos

Objetivo General

- Diseñar e implementar un entorno local automatizado para el aprovisionamiento y despliegue de servicios HTTP con Apache Web Server, integrados en un dominio DNS gestionado con Bind9, empleando infraestructura virtualizada y técnicas básicas de automatización que simulen el comportamiento de una plataforma real de servicios en la nube.

Objetivos Específicos

- Comprender el funcionamiento de un servidor DNS y desplegar Bind9 para gestionar los nombres de host de la zona local, incluyendo el registro automático de nuevos hosts.
- Desarrollar mecanismos de aprovisionamiento y gestión del servicio, automatizando la creación, configuración y eliminación de instancias del servidor HTTP.
- Automatizar el despliegue de páginas o aplicaciones web solicitadas por el usuario.

Requisitos del Proyecto

1. Infraestructura Virtualizada

- Diseño de la arquitectura general.
- Componentes involucrados: VirtualBox, red local, dominio DNS, instancias Apache.
- Interacción entre los elementos.

2. Servidor DNS (Bind9)

- Instalación y configuración de Bind9 en una VM dedicada.
- Creación de la zona DNS local ([grid.lab](#)).
- Habilidad del registro dinámico de nuevos hosts.
- Validación de la resolución de nombres hacia las IPs de las instancias Apache.

3. Servidor HTTP (Apache)

- Instalación de Apache en Debian 13 CLI.
- Configuración de red y acceso remoto.
- Preparación del disco en modo multiconexión como plantilla base.
- Requisitos de cada instancia: hostname único, directorio web propio, despliegue desde archivo [.zip](#).

4. Aplicación Web de Gestión (Golang)

- Recepción de solicitudes de aprovisionamiento.
- Ejecución de comandos remotos para crear/configurar VMs.
- Actualización automática del servidor DNS.
- Despliegue del contenido web del usuario.
- Visualización de instancias activas en un dashboard con enlaces de acceso.

5. Automatización

- Desarrollo de scripts o herramientas para automatizar todo el proceso.
- Eliminación de la intervención manual en la creación, configuración y despliegue.

Metodología

Describe el enfoque seguido para el desarrollo del proyecto. Puedes incluir:

- Herramientas utilizadas:

- VirtualBox
- Debian
- Apache
- Bind9
- Go
- html
- Python
- Php

- Configuración de red:

- Modo puente:
- Red interna:

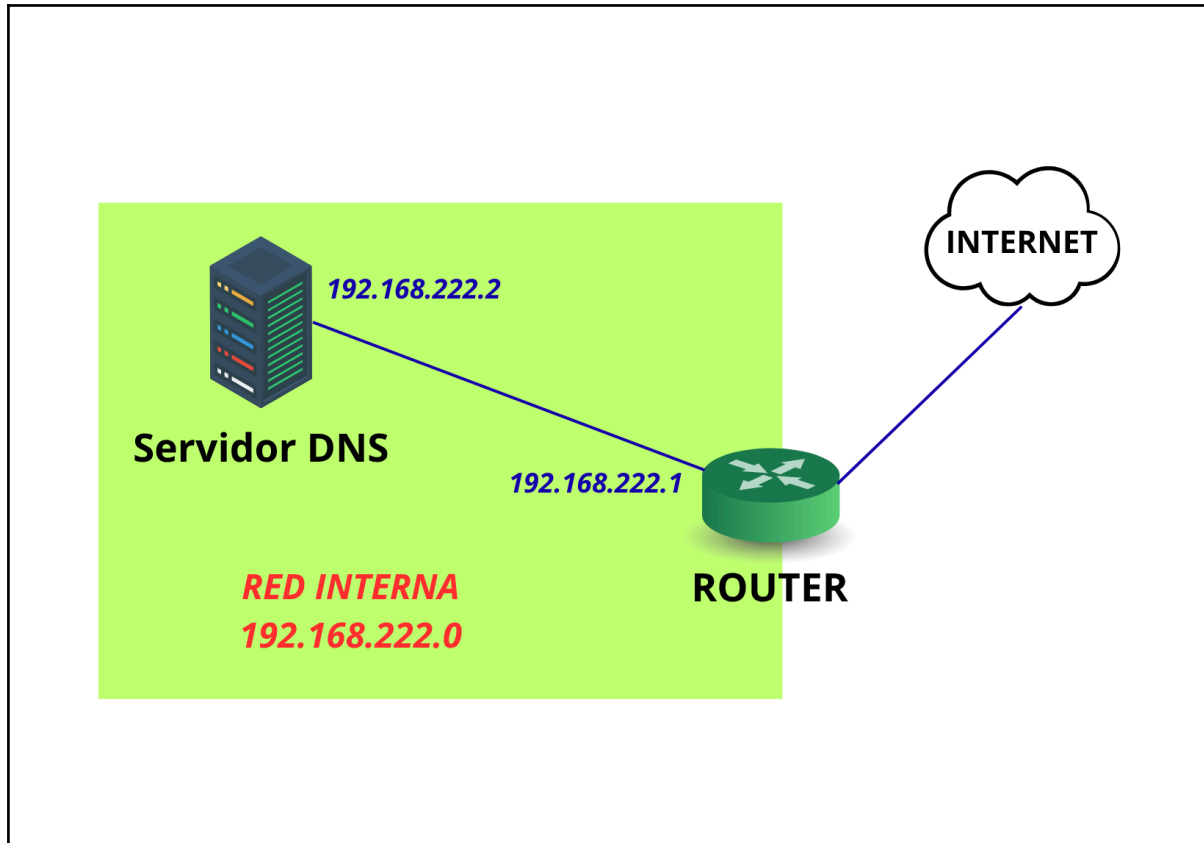
- Flujo de trabajo:

-

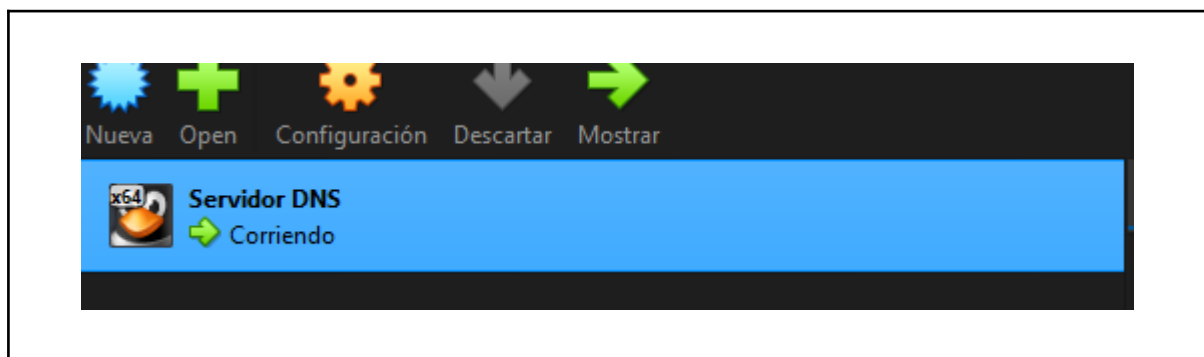
Desarrollo

1. Instalación y Configuración de Bind9

- Tipología de red



- Creación de la máquina virtual en modo puente, para realizar las instalaciones necesarias



- Instalación de Bind9

```
root@DNS:~# apt-get update
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://security.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://security.debian.org/debian-security trixie-security/main Sources [89.9 kB]
Get:5 http://security.debian.org/debian-security trixie-security/main amd64 Packages [67.8 kB]
Get:6 http://security.debian.org/debian-security trixie-security/main Translation-en [44.6 kB]
Fetched 293 kB in 3s (114 kB/s)
Reading package lists... Done
root@DNS:~# apt-get install bind9
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bind9 is already the newest version (1:9.20.15-1~deb13u1).
0 upgraded, 0 newly installed, 0 to remove and 123 not upgraded.
root@DNS:~# |
```

- Configuración de la ip estática **sudo nano /etc/network/interfaces**

```
GNU nano 8.4 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

auto enp0s3
iface enp0s3 inet static
    address 192.168.222.2
    netmask 255.255.255.0
    gateway 192.168.222.2
    dns - nameservers 192.168.222.2
    dns - search redesplus.es
```

- Configuración del archivo **/etc/bind/named.conf.local**. Ahí definimos el nombre de zona de búsqueda directa.

```
GNU nano 8.4 /etc/bind/named.conf.local *
//
// Do any local configuration here
//
// se incluye la contraseña que permitira comunicacion entre GO y Bind
include "/etc/bind/rndc.key";

zone "cloud.lab" {
    type master;
    file "/etc/bind/db.cloud.lab";
    allow-update { key "rndc-key"; }; //actualizaciones dinamica desde GO
};
```

- Luego, se configura el archivo **/etc/bind/db.lab.local**, ahí se definirán los registros tipo A records que resuelven los nombres de hosts a sus respectivas direcciones IP. Definimos el del DNS y luego se irán registrando los de las demás máquinas que se vayan creando.

```
GNU nano 8.4
$TTL 604800
@ IN SOA dns.lab.local. root.lab.local. (
    2 ; serial
    644800 ; Refresh
    86400 ; Retry
    2419200 ; Expire
    604800 ) ; Negative Cache TTL

@ IN NS dns.lab.local.
dns IN A 192.168.222.2
web1 IN A 192.168.222.3
```


- Se configuró el archivo **/etc/bind/named.conf.options**

```
GNU nano 8.4
options {
    directory "/var/cache/bind";
    listen-on {127.0.0.1; 192.168.222.2;};
    allow-query {any;};
    forwarders{
        192.168.222.1;
    };
    recursion yes;
    dnssec-validation auto;
};
```

- Se configuró el servidor DNS predeterminado **sudo nano /etc/resolv.conf** y después se protege con **sudo chattr +i /etc/resolv.conf** y se reinicia la máquina

```
GNU nano 8.4
nameserver 192.168.222.2
```

- Se verificó que los servicios estuvieran corriendo, salia un error pero se soluciona reiniciando la máquina

```
root@DNS:~# systemctl status bind9
• named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-11-01 18:23:12 -05; 47min ago
  Invocation: 45495a7b04644f1791a8e578b5f1c4d4
     Docs: man:named(8)
    Main PID: 1170 (named)
   Status: "running"
     Tasks: 5 (limit: 2297)
    Memory: 9.6M (peak: 11.8M)
       CPU: 9.107s
    CGroup: /system.slice/named.service
           └─1170 /usr/sbin/named -f -u bind
```

```
root@DNS:~# systemctl status networking
• networking.service - Raise network interfaces
   Loaded: loaded (/usr/lib/systemd/system/networking.service; enabled; preset: enabled)
   Active: active (exited) since Sat 2025-11-01 18:10:29 -05; 1h 0min ago
  Invocation: 7fe0cf91a2cd4ac2b28ea0af8f844379
     Docs: man:interfaces(5)
  Process: 634 ExecStart=/usr/sbin/ifup -a --read-environment (code=exited, status=0/SUCCESS)
  Process: 721 ExecStart=/bin/sh -c if [ -f /run/network/restart-hotplug ]; then /usr/sbin/ifup
    Main PID: 721 (code=exited, status=0/SUCCESS)
     Tasks: 0 (limit: 2297)
    Memory: 12.1M (peak: 29.1M)
       CPU: 396ms
    CGroup: /system.slice/networking.service
```

- se realizó la prueba de que el servidor está corriendo

```
root@DNS:~# dig web1.lab.local @192.168.222.2

; <<>> DiG 9.20.15-1~deb13u1-Debian <<>> web1.lab.local @192.168.222.2
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 59857
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: aaaa169a5afb014d010000006906a0f0d7f71afc68bf5b80 (good)
;; QUESTION SECTION:
;web1.lab.local.                IN      A

;; ANSWER SECTION:
web1.lab.local.                604800  IN      A      192.168.222.3

;; Query time: 0 msec
;; SERVER: 192.168.222.2#53(192.168.222.2) (UDP)
;; WHEN: Sat Nov 01 19:08:16 -05 2025
;; MSG SIZE rcvd: 87

root@DNS:~#
```

2. Plantilla Base de Apache

En nuestra máquina virtual Debian vamos a proceder a dejar la base para las instancias de los servicios, apoyándonos de Apache y adaptador de puente para lograr el despliegue de las instancias

- Instalación de Apache

en usuario root, al cual accedemos digitando “su” y proporcionando la contraseña del usuario root

ingresamos el comando: apt update, para actualizar librerías

```
debianapache [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
root@10:/# apt update
Get:1 http://security.debian.org/debian-security trixie-security InRelease [43.4 kB]
Hit:2 http://deb.debian.org/debian trixie InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
Get:4 http://security.debian.org/debian-security trixie-security/main Sources [89.9 kB]
Get:5 http://security.debian.org/debian-security trixie-security/main amd64 Packages [67.8 kB]
Get:6 http://security.debian.org/debian-security trixie-security/main Translation-en [44.6 kB]
Fetched 246 kB in 7s (35.7 kB/s)
All packages are up to date.
root@10:/# _
```

ingresamos el comando: apt install apache2, y a continuación la y para confirmar la descarga

```
Fetched 246 kB in 7s (35.7 kB/s)
All packages are up to date.
root@10:/# apt install apache2
Installing:
  apache2

Installing dependencies:
  apache2-bin      apache2-utils  libaprutil1-dbd-sqlite3  libaprutil1t64  libldap-common  liblua5.4-0  librtmp1  libssl2-modules  libssh2-1t64
  apache2-data     libapr1t64     libaprutil1-ldap        libcurl4t64     libldap2        libnghttp3-9  libssl2-2  libssl2-modules-db  ssl-cert

Suggested packages:
  apache2-doc      | apache2-suexec-custom  www-browser      | libssl2-modules-gssapi-heimdal  libssl2-modules-otp
  apache2-suexec-pristine  ufw                    | libssl2-modules-gssapi-mit      | libssl2-modules-ldap      | libssl2-modules-sql

Summary:
  Upgrading: 0, Installing: 19, Removing: 0, Not Upgrading: 0
  Download size: 3,522 kB
  Space needed: 11.6 MB / 8,128 MB available

Continue? [Y/n] y
Get:1 http://deb.debian.org/debian trixie/main amd64 libapr1t64 amd64 1.7.5-1 [104 kB]
Get:2 http://deb.debian.org/debian trixie/main amd64 libaprutil1t64 amd64 1.6.3-3+b1 [89.4 kB]
Get:3 http://deb.debian.org/debian trixie/main amd64 libaprutil1-dbd-sqlite3 amd64 1.6.3-3+b1 [14.2 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 libssl2-modules-db amd64 2.1.20+dfsg1-9 [19.0 kB]
Get:5 http://deb.debian.org/debian trixie/main amd64 libssl2-2 amd64 2.1.20+dfsg1-9 [57.5 kB]
Get:6 http://deb.debian.org/debian trixie/main amd64 libldap2 amd64 2.6.10+dfsg-1 [194 kB]
Get:7 http://deb.debian.org/debian trixie/main amd64 libaprutil1-ldap amd64 1.6.3-3+b1 [12.3 kB]
Get:8 http://deb.debian.org/debian trixie/main amd64 libnghttp3-9 amd64 1.8.0-1 [67.7 kB]
Get:9 http://deb.debian.org/debian trixie/main amd64 librtmp1 amd64 2.4+20151223.gitfa8646d.1-2+b5 [58.8 kB]
Get:10 http://deb.debian.org/debian trixie/main amd64 libssh2-1t64 amd64 1.11.1-1 [245 kB]
Get:11 http://deb.debian.org/debian trixie/main amd64 libcurl4t64 amd64 8.14.1-2 [391 kB]
93% [11 libcurl4t64 131 kB/391 kB 34%] 85.3
```

```
debianapache [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Setting up libssh2-1t64:amd64 (1.11.1-1) ...
Setting up libldap2:amd64 (2.6.10+dfsg-1) ...
Setting up libaprutil1t64:amd64 (1.6.3-3+b1) ...
Setting up libcups1t64:amd64 (2.14.1-2) ...
Setting up libaprutil1-ldap:amd64 (1.6.3-3+b1) ...
Setting up libaprutil1-dbd-sqlite3:amd64 (1.6.3-3+b1) ...
Setting up apache2-utils (2.4.65-2) ...
Setting up apache2-bin (2.4.65-2) ...
Setting up apache2 (2.4.65-2) ...
Enabling module mpm_event.
Enabling module authz_core.
Enabling module authz_host.
Enabling module authn_core.
Enabling module auth_basic.
Enabling module access_compat.
Enabling module authn_file.
Enabling module authz_user.
Enabling module alias.
Enabling module dir.
Enabling module autoindex.
Enabling module env.
Enabling module mime.
Enabling module negotiation.
Enabling module setenvif.
Enabling module filter.
Enabling module deflate.
Enabling module status.
Enabling module reqtimeout.
Enabling conf charset.
Enabling conf localized-error-pages.
Enabling conf other-vhosts-access-log.
Enabling conf security.
Enabling conf serve-cgi-bin.
Enabling site 000-default.
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' + '/usr/lib/systemd/system/apache2.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service' + '/usr/lib/systemd/system/apache-htcacheclean.service'.
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for libc-bin (2.41-12) ...
root@10:/# _
```

Una vez haya finalizado, podemos ver el estado de apache2 con el comando

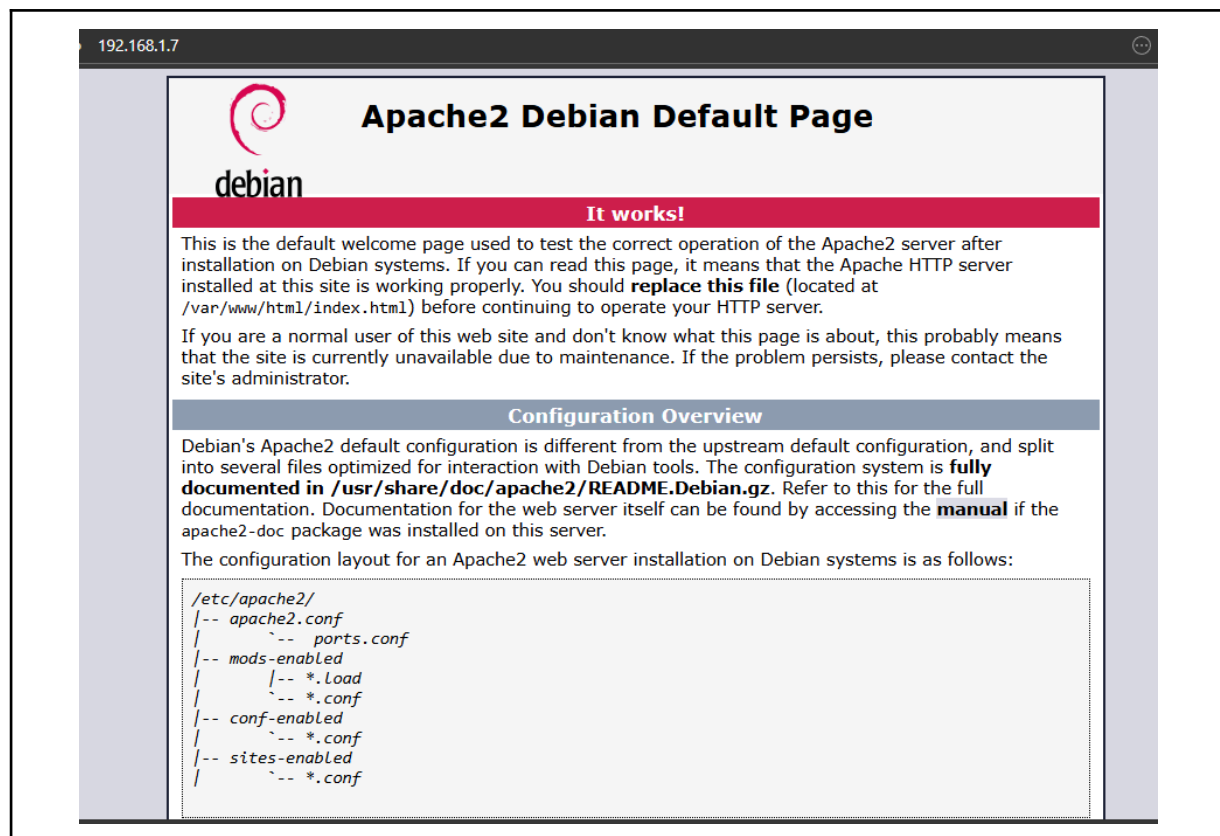
systemctl status apache2

```
debianapache [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

oot@10:/# systemctl status apache2
apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-10-29 03:28:58 EDT; 2min 37s ago
 Invocation: c543df6bedc94ffa9c33caa623ddf5f9
   Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 2100 (apache2)
    Tasks: 55 (limit: 2318)
   Memory: 5.5M (peak: 5.7M)
     CPU: 104ms
    CGroup: /system.slice/apache2.service
            └─2100 /usr/sbin/apache2 -k start
              └─2104 /usr/sbin/apache2 -k start
                └─2105 /usr/sbin/apache2 -k start

ct 29 03:28:58 10 systemd[1]: Starting apache2.service - The Apache HTTP Server...
ct 29 03:28:58 10 apachectl[2099]: AH00558: apache2: Could not reliably determine the server's fu
ct 29 03:28:58 10 systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-17/17 (END)
```

y ya tenemos apache corriendo como http



- Se crea un archivo de configuración /usr/local/bin/deploy_site.sh en la máquina base, del cual se usará el disco como multiconexion; esto es para que pueda desplegarse el programa que se pasa.

```
HOST=$1
ZIP_PATH="/tmp/sitio.zip"
DEST_DIR="/var/www/$HOST"
CONF_FILE="/etc/apache2/sites-available/$HOST.conf"

sudo unzip -o "$ZIP_PATH" -d "$DEST_DIR"

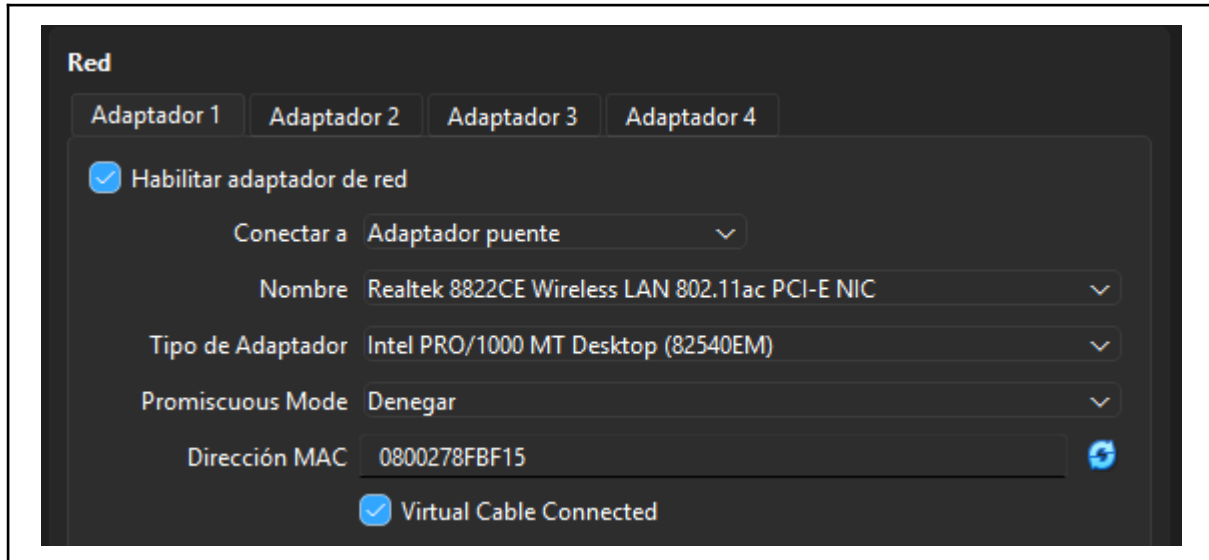
sudo tee "$CONF_FILE" > /dev/null <<EOF
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    DocumentRoot $DEST_DIR
    <Directory $DEST_DIR>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog \${APACHE_LOG_DIR}/${HOST}_error.log
    CustomLog \${APACHE_LOG_DIR}/${HOST}_access.log combined
</VirtualHost>
EOF

sudo a2ensite $HOST.conf
sudo systemctl reload apache2
sudo chown -R www-data:www-data "$DEST_DIR"
sudo chmod -R 755 "$DEST_DIR"

echo "Sitio desplegado correctamente: http://$(hostname -I | awk '{print $1}')
```

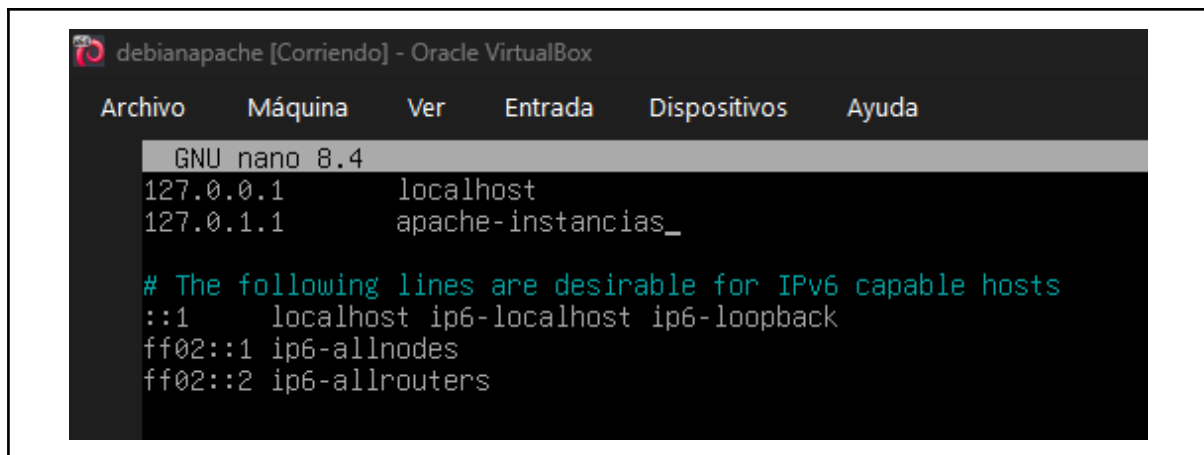
- Configuración de red y hostname

red configurada por adaptador puente



hostname configurado con los comandos

nano /etc/hosts: le ponemos el nombre que queremos al hostname



- Preparación del disco multiconexión

dentro de apache.conf apuntamos a la ruta de la carpeta donde desplegamos nuestra página y le quitamos la necesidad de contraseña a nuestro usuario

```
GNU nano 8.4 /etc/apache2/sites-available/000-default.conf
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/depoy

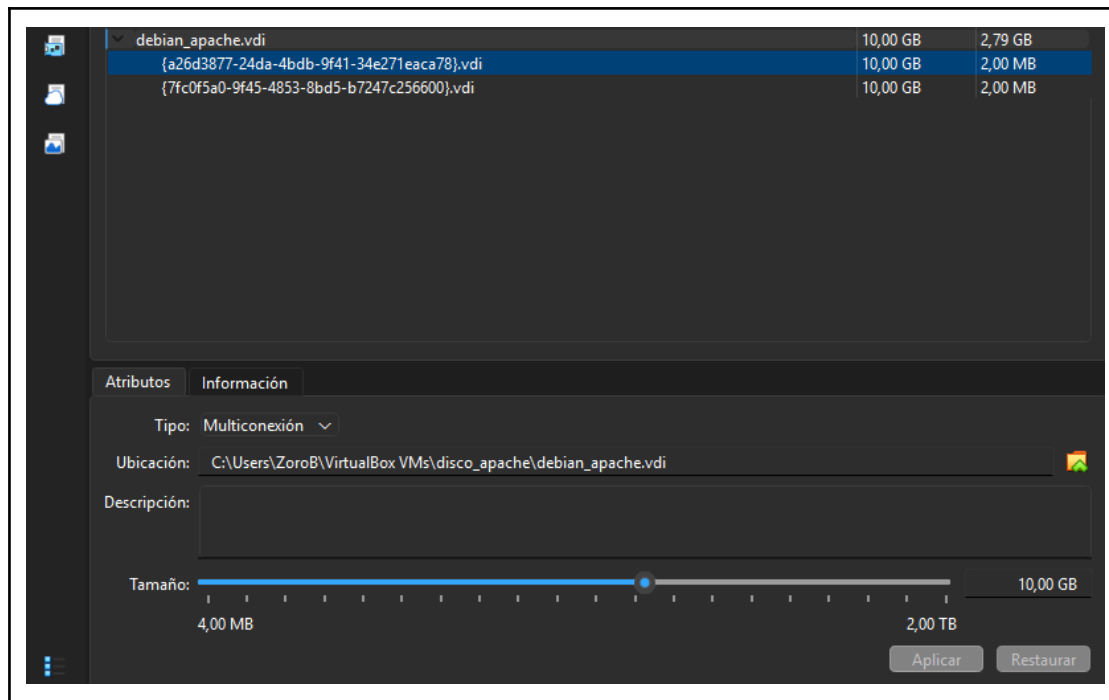
    <Directory /var/www/depoy>
        Options +ExecCGI -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

# User privilege specification
root    ALL=(ALL:ALL) ALL
vboxuser ALL=(ALL:ALL) NOPASSWD: ALL
# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:
|
@includedir /etc/sudoers.d
```

al final lo volvemos multiconexion y ya descomprimiendo los archivos en var/ww/depoy tenemos el sitio web cargado

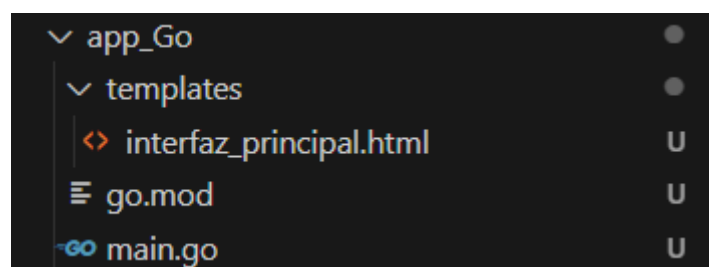


3. Automatización del Aprovisionamiento

- Scripts de clonación y configuración
- Registro dinámico en DNS
- Despliegue de contenido web

4. Desarrollo de la Aplicación Web en Go

La estructura de la aplicación web fue montada en la carpeta llamada “app_Go” que cuenta con los archivos principales de Go ([main.go](#) y [go.mod](#)) y una carpeta llamada “templates” que contiene el código html de la interfaz de usuario ([interfaz_principal.html](#)).



- Estructura del backend

Creamos el [mod.go](#)

```
PS C:\Users\ZoroB\vscodeProjects\Compunube-virtual-http-dns-lab\app_Go> go mod init proyecto_cloud
go: creating new go.mod: module proyecto_cloud
```

Resultado:

```
go.mod u x
app_Go > go.mod
Reset go.mod diagnostics | Run go mod tidy
1 module proyecto_cloud
2
3 go 1.25.3
4
```

El código del backend fue desarrollado en [main.go](#) por mediante funciones que nos permite

Cargar la página principal

```
func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        http.ServeFile(w, r, "templates/interfaz_principal.html")
    })

    http.HandleFunc("/crear", solicitudHhandler)
    http.HandleFunc("/publicar", publicarHandler)
    http.Handle("/uploads/", http.StripPrefix("/uploads/", http.FileServer(http.Dir("./uploads"))))
    http.HandleFunc("/eliminar", eliminarHandler)

    fmt.Println("Servidor escuchando en http://localhost:8088")
    http.ListenAndServe(":8088", nil)
}
```

Publicar el servicio

primero parseamos el .zip y el nombre del host para cargarlo al servicio

```

func publicarHandler(w http.ResponseWriter, r *http.Request) {
    // Parsear formulario
    err := r.ParseMultipartForm(32 << 20) // 32MB
    if err != nil {
        http.Error(w, "Error al parsear el formulario", http.StatusBadRequest)
        return
    }

    host := r.FormValue("host")
    file, handler, err := r.FormFile("contenido")
    if err != nil {
        http.Error(w, "Archivo no recibido", http.StatusBadRequest)
        return
    }
    defer file.Close()

    // Guardar archivo temporal
    tempDir := "./uploads/" + host
    os.MkdirAll(tempDir, os.ModePerm)
    filePath := filepath.Join(tempDir, handler.Filename)
    out, err := os.Create(filePath)
    if err != nil {
        http.Error(w, "Error al guardar archivo", http.StatusInternalServerError)
        return
    }
    defer out.Close()
    io.Copy(out, file)
    fecha := time.Now().Format("2006-01-02 15:04:05") //de prueba
}

```

creamos la máquina virtual y obtenemos la ip

```

// Solicitar maquina virtual por host y obtener el ip
cmd := exec.Command("cmd", "/C", "Scripts\\library\\New_VM.py")
var out_bytes bytes.Buffer
cmd.Stdout = &out_bytes // capturamos errores
cmd.Stderr = &out_bytes
error := cmd.Run()
if error != nil {
    fmt.Println("error creando maquina virtual", err)
}
fmt.Println(" Salida del script Python:\n", out_bytes.String())
time.Sleep(30000000000)

// Obtener la salida y limpiar espacios
raw := out_bytes.String()
raw = strings.TrimSpace(raw)

// Buscar la primera IP válida en la salida
re := regexp.MustCompile(`\b\d{1,3}(\.\d{1,3}){3}\b`)
ip := re.FindString(raw)

if ip != "" {
    fmt.Println(" IP detectada:", ip)
    // Aquí puedes guardarla en una variable, archivo, base de datos, etc.
} else {
    fmt.Println(" No se detectó una IP válida. Salida:", raw)
}

fmt.Println(ip)
despliegue(filePath, ip)

```

después de desplegarlo dentro de la máquina virtual

instanciamos el frontend

```

// Responder al frontend
instancia := Instancia{
    Host: host,
    IP: ip,
    URL: url,
    Fecha: fecha,
}
w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(map[string]interface{}{
    "exito": true,
    "host": instancia.Host,
    "ip": instancia.IP,
    "url": instancia.URL,
    "fecha": instancia.Fecha,
})
}

```

- Interfaz de usuario

La interfaz de usuario se desarrollo con html y javascript de la siguiente manera

definimos la cabecera y los estilos de nuestra página web

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Gestión de Servicios Web</title>
6   <style>
7     body {
8       font-family: Arial, sans-serif;
9       background-color: #eef2f5;
10      padding: 40px;
11    }
12    .container {
13      background-color: #fff;
14      padding: 30px;
15      border-radius: 8px;
16      max-width: 600px;
17      margin: auto;
18      box-shadow: 0 0 10px rgba(0,0,0,0.1);
19    }
20    h2 {
21      text-align: center;
22      margin-bottom: 30px;
23    }
24    label {
25      display: block;
26      margin-top: 20px;
27      font-weight: bold;
28    }
29    input[type="text"],
30    input[type="file"] {
31      width: 100%;
32      padding: 10px;
33      margin-top: 8px;
34      border: 1px solid #ccc;
35      border-radius: 4px;
36    }
37    button {
38      margin-top: 20px;
39      padding: 10px 20px;

```

```

37    button {
38      margin-top: 20px;
39      padding: 10px 20px;
40      background-color: #0078D7;
41      color: white;
42      border: none;
43      border-radius: 4px;
44      cursor: pointer;
45    }
46    button:hover {
47      background-color: #005fa3;
48    }
49    .dashboard {
50      margin-top: 40px;
51    }
52    table {
53      width: 100%;
54      border-collapse: collapse;
55      margin-top: 10px;
56    }
57    th, td {
58      padding: 10px;
59      border: 1px solid #ccc;
60      text-align: left;
61    }
62    th {
63      background-color: #f0f0f0;
64    }
65    a {
66      color: #0078D7;
67      text-decoration: none;
68    }
69  </style>
70 </head>
71 <body>

```

Definimos el cuerpo de la página

```

app_Go > templates > <> interfaz_principal.html > html > body > script > eliminarFila > body > host
 2  <html lang="es">
71  <body>
72    <div class="container">
75      <label for="host">Nombre del host</label>
76      <input type="text" id="host" name="host" placeholder="Ej. paisajes">
77
78      <button onclick="crear()"> aceptar </button>
79
80      <label for="contenido">Contenido web (comprimido)</label>
81      <input type="file" id="contenido" name="contenido" accept=".zip,.tar.gz">
82
83      <button onclick="publicar()">Publicar servicio</button>
84
85      <div class="dashboard">
86        <h3>Registro de actividad</h3>
87        <table id="tablaInstancias">
88          <thead>
89            <tr>
90              <th>Host</th>
91              <th>IP</th>
92              <th>Enlace</th>
93              <th>Fecha de creación</th>
94              <th></th>
95            </tr>
96          </thead>
97          <tbody>
98            <!-- Se llenará dinámicamente -->
99          </tbody>
100        </table>
101      </div>
102    </div>

```

Definimos el contenido de script para conectarnos a nuestro backend

```

127  }
128  async function publicar() {
129    const host = document.getElementById('host').value;
130    const archivo = document.getElementById('contenido').files[0];
131
132    if (!host || !archivo) {
133      alert("Por favor ingresa un nombre de host y selecciona un archivo.");
134      return;
135    }
136    const formData = new FormData();
137    formData.append("host", host);
138    formData.append("contenido", archivo);
139    try {
140      const response = await fetch("/publicar", {
141        method: "POST",
142        body: formData
143      });
144
145      const resultado = await response.json();
146      if (resultado.exito) {
147        agregarInstancia(resultado.host, resultado.ip, resultado.url, resultado.fecha);
148      } else {
149        alert("Error al aprovisionar: " + resultado.mensaje);
150      }
151    } catch (error) {
152      console.error("Error:", error);

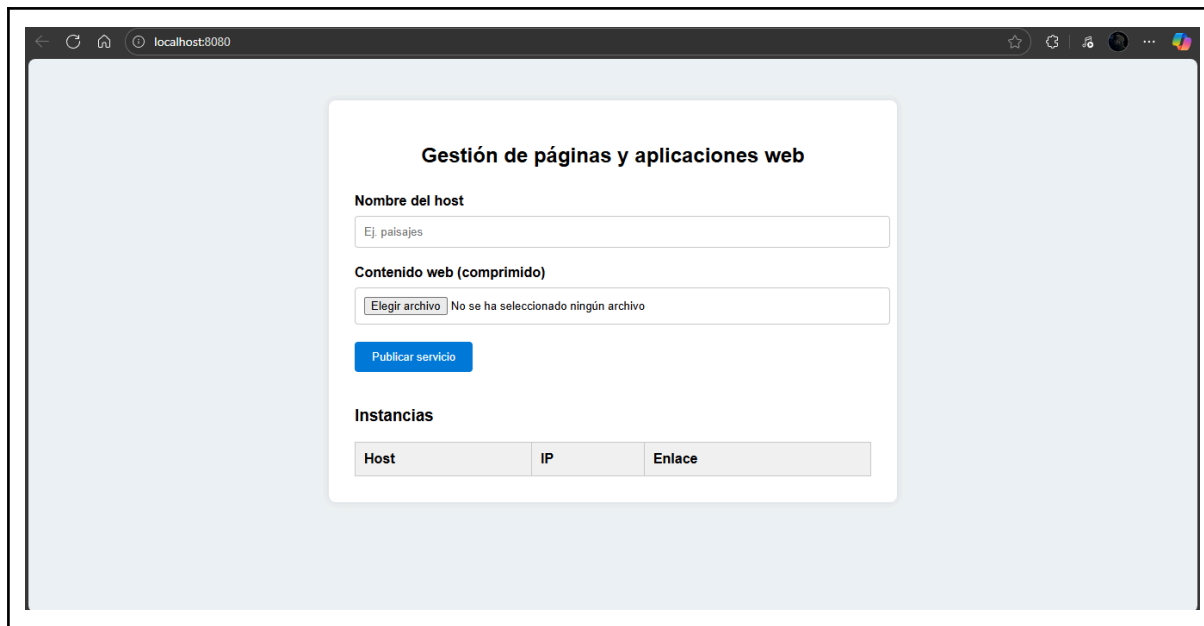
```

```

153     alert("No se pudo conectar con el servidor.");
154 }
155 }
156
157 function agregarInstancia(host, ip, url, fecha) {
158     const tabla = document.getElementById("tablaInstancias").querySelector("tbody");
159     const fila = document.createElement("tr");
160
161     fila.innerHTML = `
162         <td>${host}</td>
163         <td>${ip}</td>
164         <td><a href="${url}" target="_blank">Ver página</a></td>
165         <td>${fecha}</td>
166         <td> <button onclick="eliminarFila(this)"> Eliminar </button> </td>
167     `;
168
169     tabla.appendChild(fila);
170 }
171
172 function eliminarFila(boton) {
173     const fila = boton.closest("tr");
174     const host = fila.cells[0].textContent;
175     const ip = fila.cells[1].textContent;
176     const url = fila.cells[2].querySelector("a").href;
177
178     // Aquí puedes hacer lo que necesites con los datos
179     console.log("Eliminando instancia:");
180     console.log("Host:", host);
181     console.log("IP:", ip);
182     console.log("URL:", url);
183
184     // Eliminar visualmente la fila
185     fila.remove();
186
187     // Opcional: enviar solicitud al backend para eliminar la instancia
188     fetch("/eliminar", {
189         method: "POST",
190         headers: { "Content-Type": "application/json" },
191         body: JSON.stringify({ host, ip, url })
192     }).then(res => res.json())
193     .then(data => console.log("Respuesta del servidor:", data));
194
195 }
196 </script>
197 </body>

```

Resultado final:



- Integración con VirtualBox y Bind9

Conclusiones

- **Reflexión sobre los aprendizajes obtenidos.**

Este proyecto nos permitió integrar conocimientos teóricos de computación en la nube con aplicaciones prácticas, que permitieron consolidar nuestra comprensión en:

- Como funcionan los modelos IaaS y PaaS desde la perspectiva del proveedor de servicios.
- Aprendimos a orquestar múltiples componentes (virtualización, DNS, servidores web) mediante scripts y aplicaciones personalizadas.
- Aprendimos el funcionamiento de DNS y su importancia en los entornos de servicios distribuidos.
- Dominamos el uso de discos en modo multiconexión en VirtualBox, una técnica importante para la clonación eficiente de máquinas virtuales.

- **Dificultades enfrentadas y cómo se resolvieron:**

1. **Configuración de Bind9**

- Dificultad: Era un software nuevo para nosotros, especialmente la configuración de zonas DNS dinámicas y las actualizaciones automáticas.
- Solución: Investigamos documentación oficial y tutoriales específicos, implementamos scripts de actualización DNS que se ejecutan desde la aplicación Go mediante SSH.

2. **Comunicación entre componentes**

- Dificultad: Coordinar la aplicación de Go con VirtualBox, Apache y Bind9 de manera automatizada.
- Solución: Desarrollamos una arquitectura modular con interfaces claras entre componentes, usando ejecución remota de comandos y transferencia de archivos.

- **Posibles mejoras o extensiones futuras del proyecto.**

- Adaptar el sistema para funcionar con proveedores de nube pública (como AWS, Azure o Google Cloud), aprovechando sus herramientas de gestión y de almacenamiento distribuido.
- implementar control de acceso para usuarios con diferentes niveles de permisos
- Migrar de máquinas virtuales a contenedores para un aprovisionamiento más rápido y eficiente.

Bibliografía

- Redes Plus. (2019). DNS LINUX INSTALAR y configurar BIND9 Ubuntu. https://youtu.be/hq8kWcpGUvw?si=_dHsnHDe2xqX053p
- Microsoft. (2025). *Copilot* [Asistente de inteligencia artificial]. Microsoft. <https://copilot.microsoft.com>
- Google. (s. f.). *Standard library* | *Go packages*. Recuperado el 29 de octubre de 2025, de <https://pkg.go.dev/std>
- **CyberCiti.biz. (2022).** *How to change hostname on Debian 10 Linux*. Recuperado el 29 de octubre de 2025, de <https://www.cyberciti.biz/faq/how-to-change-hostname-on-debian-10-linux/>
- OpenAI. (2025). Script de despliegue automático de sitios web en Apache [Código generado por ChatGPT]. <https://chat.openai.com/>
- **Microsoft Corporation.** (2024). *ARP Command Reference*. Windows Commands Documentation. <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/arp>
- **Canonical Ltd.** (2024). *OpenSSH Server Configuration*. Ubuntu Server Documentation. <https://ubuntu.com/server/docs/service-openssh>

Anexos

- Enlace al repositorio de GitHub

<https://github.com/juandajedrez/-Compunube-virtual-http-dns-lab.git>